

W4 array (pointer)

2024年12月30日 19:25

注:

1.用变量定义数组长度时, 不能同时初始化。

##

指针数组和数组指针

指针数组: 储存指针

Int *a[10]; (等价于int *(a[10]);)

a不是指针

数组指针:

Int (*p)[3];

指向列数为3的二维数组的指针变量

##

函数与指针

函数原型:

```
int sum ( int *arr , int n ) ;  
int sum ( int * , int n ) ;  
int sum ( int arr[ ] , int n ) ;  
int sum ( int [ ] , int n ) ;
```

函数定义:

```
int sum ( int *arr , int n ) ;  
int sum ( int arr[ ] , int n ) ;
```

多维数组

```
int sum ( int a[][4] , int r ) ;
```


冒泡排序（Bubble Sort）
平均时间复杂度：O(n^2)
最好情况时间复杂度：O(n)
最坏情况时间复杂度：O(n^2)

通过重复地遍历待排序序列，比较相邻元素的大小并交换它们的位置，直到没有元素需要交换为止。

最坏情况：序列完全逆序，冒泡排序需要进行n-1轮比较，每轮比较都需要遍历整个序列
最好情况：序列已经有序，冒泡排序只需要进行一轮比较

选择排序（Selection Sort）
平均时间复杂度：O(n^2)
最好情况时间复杂度：O(n^2)
最坏情况时间复杂度：O(n^2)

在每一轮迭代中选择剩余元素中的最小（或最大）元素，并将其放到序列的起始位置

选择排序的时间复杂度与输入序列的顺序无关，因为每轮迭代都需要遍历剩余元素以找到最小（或最大）元素

插入排序（Insertion Sort）
平均时间复杂度：O(n^2)
最好情况时间复杂度：O(n)
最坏情况时间复杂度：O(n^2)

通过构建有序序列，对于未排序数据，在已排序序列中从后向前扫描，找到相应位置并插入。

最坏情况：序列完全逆序，每次插入都需要将已排序的元素逐个向后移动
最好情况：序列已经有序，只需要遍历一次序列即可完成
平均情况：时间复杂度也是O(n^2)，但通常比冒泡排序和选择排序稍快一些。

希尔排序（Shell Sort）
平均时间复杂度：O(n log n) 到 O(n^2)，取决于增量序列的选择
最好情况时间复杂度：O(n log n)
最坏情况时间复杂度：O(n^2)

也称递减增量排序算法，是插入排序的一种更高效的改进版本，是非稳定排序算法

先将整个待排序的记录序列分割成为若干子序列分别进行直接插入排序，待整个序列中的记录“基本有序”时，再对全体记录进行依次直接插入排序。



具体算法步骤为：
选择一个增量序列t1, t2, ..., tk, 其中ti>tj, tk=1。
按增量序列个数k，对序列进行k趟排序。
每趟排序，根据对应的增量ti，将待排序列分割成若干长度为m的子序列，分别对各子表进行直接插入排序。

希尔排序的时间复杂度与增量序列的选择有关，但通常情况下，希尔排序的时间复杂度较直接插入排序、冒泡排序、选择排序等方法有较大改进。在实际应用中，希尔排序是一种性能较好的排序算法。

归并排序（Merge Sort）
平均时间复杂度：O(n log n)
最好情况时间复杂度：O(n log n)
最坏情况时间复杂度：O(n log n)

采用分治策略，将序列递归地分成两半，直到子序列长度为1（认为已有序），然后将有序子序列合并成一个有序序列

时间复杂度与输入序列的顺序无关，总是为O(n log n),因为它将问题划分为两个大致相等的子问题，并将它们递归地解决，然后将结果合并

快速排序（Quick Sort）
平均时间复杂度：O(n log n)
最好情况时间复杂度：O(n log n)
最坏情况时间复杂度：O(n^2)（当输入数据已经排序或逆序时）

一种高效的排序算法，它采用分治策略,通过选择一个基准元素，将序列划分为左右两部分，左边部分小于基准，右边部分大于基准，然后递归地对左右两部分进行排序

最好情况：每次划分都能将序列均匀分为两部分
最坏情况：输入序列已经有序或逆序时
平均情况：时间复杂度为O(n log n)，但由于划分的不均匀性，实际性能可能有所波动。

W7 recursion

2024年12月30日 21:30

##

递归的4个基础

Base Case: 至少存在一种情况, 不再进一步递归就可以直接得到结果

Make Progress: 每一次递归都是向着base case前进

Always Believe: 始终相信递归是可行的

Compound Interest Rule: 不要在不同的递归调用中做相同的计算

##

线性递归 vs 树状递归

阶乘: 线性递归 (每一次递归只调用一次自己)

斐波那契数列: 树状递归 (每一次递进要调用两次自己)

树状递归通常存在大量的重复计算

记忆是消除重复计算的主要手段

##

尾递归

递进时不对递进调用返回的值再做任何计算就直接返回

return gcd(y, x%y);

尾递归是伪递归, 因为递进的时候做了计算, 而回归的时候没有做计算

所有的尾递归都可以在代码形式上机械地被优化为非递归的形态:

整个函数用 while (1) 包起来

base case改成 if ... break

递进改成对相应的变量赋值

##

迭代算法vs递归算法

简单地可以认为用循环实现的是迭代算法

迭代算法: 从问题最小的状态开始, 展开到最大的状态

$n! = 1 * 2 * 3 * \dots * n$

递归算法: 从问题最大的状态开始, 分解到最小的状态 (base case)

$n! = n * (n-1) * (n-2) * \dots * 1$

绝大部分递归算法都可以被改造成迭代算法

机械的方法: 将结果作为一个参数, 在递进的过程中计算并传递进下一轮递

进, 从而把递归改造成尾递归

递归:

```
int factorial(int n) {
    if ( n==0 ) {
        return 1;
    }
    return n*factorial(n-1);
}
```

迭代:

```
int factorial_it(int n, int f) {
    if ( n==0 ) {
        return f;
    }
    return factorial_it(n-1, f*n);
}
```

##

Pros :

一旦被优化, 性能还不错

代码更简短

易于理解: 递归代码更有描述性, 比迭代代码更接近问题本身

Cons:

递归深度受到线程运行栈大小限制不能太深

占用内存较多

建议: 用递归计算的思路思考问题, 代码实现后, 转成迭代实现

e.g.

Euclid算法 (求最大公约数)

```
int gcd(int x, int y) {
    if ( y==0 ) {
        return x;
    }
    return gcd(y, x%y);
}
```

汉诺塔问题:

在一根柱子上从下往上按照大小顺序摆着64片黄金圆盘, 把圆盘从下面开始按大小顺序重新摆放在另一根

柱子上。并且规定, 在小圆盘上不能放大圆盘, 在三根柱子之间一次只能移动一个圆盘

```
void hanoi(int n, char source, char target, char aux) {
    if ( n>0 ) {
        hanoi(n-1, source, aux, target); // 以target为辅助柱
        printf("move %d from %c to %c\n", n, source, target);
        hanoi(n-1, aux, target, source);
    }
}
```

加速幂计算:

```
int power(int x, int n) {
    if (n==0) {
        return 1;
    } else if (n%2==1) {
        return x * power(x, n-1);
    } else {
        int t = power(x, n/2);
        return t*t;
    }
}
```

W10 pointer and string

2024年12月30日 8:42

& can't be applied to values that don't have addresses
e.g.

&(a+b) &(a++) &(++a)

```
##
字符串定义
char *str ="Hello";
char word[] ="Hello";
char line[10] ="Hello";
char *p; p="string";
错误: char str[10]; str="string";
```

```
#
数组和指针定义字符串
数组:
这个字符串在这里
作为本地变量空间自动被回收
指针:
这个字符串不知道在哪里
处理参数
动态分配空间
如果要构造一个字符串—>数组
如果要处理一个字符串—>指针
```

```
##
Escape char
用来表达无法印出来的控制字符或特殊字符,
它由一个反斜杠 \ 开头, 后面跟上另一个字
符, 这两个字符合起来, 组成了一个字符( \
back-slash itself )
```

char	meaning
\b	back position
\t	next table stop
\"	double quote
\'	single quote
\n	new line
\r	return the carriage

@字符串基本内容
字符串是以空字符 (‘\0’) 结尾的 char 类型数组。
注: 双引号: 字符串; 单引号: 单个字符。
如: ‘ x’ 是一个字符; “ x” 是一个字符串 (实际上由两个字符 ‘ x’ 、 ‘ \0’ 组成)

```
# 单字符的输入和输出 #:
getchar()(读取但不存储输入; 包括 ‘\n’ )
putchar(char a);
返回写了几个字符, EOF (-1) 表示写失败
```

```
@ 字符串函数
<string.h>
1. 读取函数:
scanf() 注: 在遇到 第一个空白 (空格、制表符、换行符) 时不再读取输入
gets() 读取 整行输入, 直至换行符 (不读入换行符)
注: 可能会溢出, 所以用 fgets() 代替
gets_s() 会读取换行符但是丢弃它; 不需要 fgets() 的第三个参数
```

```
##
fgets() 会将 换行符读入并存储; 包含三个参数 (指针或数组, 读入字符的最大
数量 (实际读入 n-1 个) , 要读入的文件 (如果读入从键盘输入的数据, 则以
stdin 作为参数) ); 与 fputs() 配合使用; 如果读到文 件结尾则返回 NULL
```

```
2. 计算长度函数:
strlen() 计算 包括空格和标点符号在内的字符数
size_t strlen(const char *s);
注: (1) 区分 sizeof 运算符: 如 char a[10]=" string." ;sizeof(a)=
10;strlen(a)=7;
(2)sizeof 与 strlen() 返回值对应 %zd
```

```
3.字符串连接的函数
char *strcat(char *dest, const char *src);
作用: 把两个字符数组中的字符串连接起来, 把字符串2连接到字符串1的后面, 结果放在字符数组1中
一定保证字符串1有足够的空间!!!
```

```
4.字符串复制的函数
char *strcpy(char *dest, const char *src);
作用: 将字符串2复制到字符数组1中去
strcpy(字符数组1, 字符串2, n) (选择复制)
n: 表示将字符串2中的n个单个字符复制到字符数组1中, 最少为0个, 最多不
能超过字符串2的长度
```

```
5.字符串比较的函数
strcmp(字符数组1, 字符串2)
因为字符串不能用等号来比较大小, 所以就用strcmp函数来比较
比较规则:
(1) 如果全部字符相同, 则认为两个字符串相等, 返回0;
(2) 若出现不相同的字符, 则以第一对不相同的字符比较结果为准
('a'<'z'; 'A'<'Z') ;
如果字符串1 > 字符串2, 则函数值返回一个正数; 如果字符串1 < 字符串2,
则函数值返回一个负数。
strncmp(字符数组1, 字符串2, n) (选择比较)
n: 选择字符串的前n个字符进行比较
```

```
6. 输出函数:
puts() 注: 遇到空字符时停止; 只能输出字符串, 而且 自动在显示的字符串末尾加上换行符; 如果与fgets() 混合使用, 则末尾会有两个换行符
fputs() 含两个参数 (输出的内容, 要写入的文件 (如果显示在显示器上, 使用
stdout 作为参数) ); 末尾不添加换行符
printf() 输出 多个字符串更加方便
```

```
7.其他函数
strlwr(字符串)——转换为小写的函数
作用: 将字符串中的大写字母转换成小写字母
strupr(字符串)——转换为大写的函数
作用: 将字符串中的小写字母转换成大写字母
```

@字符串相关的处理方法
1. 处理换行符的方法:

注:
pointer
1.a为指针, a[i]= * (a+i)
2.*p++
++和*的优先级相同, 从右向左运算
3.无论指向什么类型, 所有的指针的大小都一样, 因为都是地址
但是指向不同类型数据的指针不能直接互相赋值的, 为了避免用错指针

```
String
1.0标志字符串的结束, 但不是字符串的一部分
计算字符串长度时不包含末尾的0
2.char* s = "Hello, world!";
s实际上是 const char* s
不能对s所指的字符串做写入和修改
不能之后写s[0]='H';
注意:
当指针直接指向字符串常量时, 字符串常量通常存储在只读数据段, 不
能通过指针去修改字符串常量的内容 (否则会导致运行时错误, 如段错
误)
而对于指向动态分配内存的指针字符串, 其存储的字符串在动态分配的
堆空间中, 可以根据实际情况进行修改 (只要操作合法且在分配的内存
范围内)
```

```
如果需要修改字符串, 应该用数组
char s[] = "Hello, world!";
3.正确:
char *p;
p=(char *)malloc(10);
scanf("%s", p);
错误:
char *p;
scanf("%s", p);
没有初始化!!!!
```

- (1) 如 `while ( words[i]!=' \n' )i++;words[i]='\0' ;`
- (2) 丢弃多出的字符串: 如 `while(getchar()!=' \n' )continue;`

W14 program structure

2024年12月30日 22:10

#标准头文件结构#

```
#ifndef __A_H__
#define __A_H__
#include "node.h"
typedef struct _list {
    Node *head;
    Node *tail;
}
#endif
```

运用条件编译和宏，保证这个头文件在一个编译单元中只会被 #include 一次
#pragma once 也能起到相同的作用，但是不是所有的编译器都支持

##注：

- 1.#开头的编辑预处理指令 **不是C的语句**，没有结尾的分号
- 2.编译预处理程序 (cpp) 终端指令，保留编译链接过程中的中间过程文件：
gcc -save-temps a.c
多个.c文件编译：gcc a.c b.c
一个.c文件是一个编译单元，编译器每次编译只处理一个编译单元
- 3.如果一个宏的值超过一行，最后一行之前的行末需要加 \
宏的值后面出现的注释不会被当作宏的值的一部分
- 4.#define cube(x) ((x)*(x))

```
错误定义的宏：
#define RADTODEG(x) (x * 57.29578)
#define RADTODEG(x) (x) * 57.29578
5.#define PRINT_INT(n) printf(#n "=%d", n)
相当于printf( "n" "=%d", n)
7.## operator：相当于紧挨着
8.#error "my bad"
9.头文件a.h
写入函数的原型
```

一般的做法:任何.c都有对应的同名的.h，把所有对外公开的函数的原型和全局变量的声明都放进去
#include "a.h" //文本插入 (不是用来引入库)
""：先在当前目录中寻找，找不到再跳到指定目录
<>：只在编译器指定的目录中寻找

注：不对外公开的函数：在函数前面加上 static 就使得它成为只能在所在的编译单元中被使用的函数
10.没有值的宏

#ifdef (如果已定义) 是一个预处理指令，用于条件编译。它的作用是检查一个宏是否已经被定义，如果被定义了，则编译器会编译#ifdef和相应的 #endif之间的代码块。这是处理不同编译环境或平台之间代码差异的一种常用方法。
常见用途：防止重复包含头文件：如果 HEADER_FILE_H'没有被定义，那么头文件的内容会被包含；如果已经被定义了，那么内容不会被再次包含。

#变量的储存类型#

@分类：

静态存储：存储静态变量；在程序编译时就已经分配好内存空间，该变量一直占有固有的内存空间，程序结束后才释放该部分内存空间
动态存储：储存局部变量；在程序运行过程中，只有当变量所在的函数被调用时候，该变量才会被分配内存空间，当被调函数调用结束时候，该变量内存空间就会被释放，值就会消失 (动态存储区：堆)

@变量储存类型声明

自动变量声明auto
函数中的局部变量如果不用static声明，默认为自动变量；
储存在动态区；

静态变量声明static

储存在静态区；
分类：
<1>静态局部变量：在调用到该函数时分配内存空间，但是当函数调用结束后并不会释放该变量内存空间，而是保存起来，下一次调用还是上一次的值
<2>静态全局变量：静态全局变量 **只能在本源文件**调用，不能被其他源文件调用 (一个程序一般由多个源文件组成)

寄存器变量声明register

存储空间也是动态分配的，调用结束后便释放空间；
目的是建议编译器将该变量存储在 CPU 的寄存器中，而不是像普通变量那样存储在内存里。因为 寄存器的访问速度比内存的访问速度快，若某变量需要频繁访问，就可以定义为寄存器变量，将大大提高程序的执行效率

外部变量声明extern

在 **另一个源文件** 中想要使用该变量就可以用extern声明 (前提是该变量 **不是静态变量**) ；
举例来说，如果文件a.c需要引用b.c中变量int v，就可以在a.c中声明extern int v，然后就可以引用变量v

归纳

变量类型	自动局部变量	静态局部变量	静态全局变量	非静态全局变量	寄存器变量
存储区	动态区	静态区	静态区	静态区	寄存器
函数结束变量的值	消失	保存	保存	保存	保存
未初始化的变量的值	不确定	0	0	0	不确定
作用域	本函数	本函数	本文件	整个程序	本函数

##静态全局变量和普通全局变量

作用域
静态全局变量： 仅限于定义它的源文件，其他源文件不能直接访问这个静态全局变量 (有助于避免不同源文件中的同名变量相互干扰，提高了程序的模块化性)
普通全局变量： 整个程序，一旦定义了一个全局变量，在同一个程序的所有源文件中，只要经过适当的声明 (extern)， 就可以访问这个全局变量

生命周期

都是整个程序的运行期间。在程序开始运行时分配内存空间，在程序结束时释放内存，即使在函数调用过程中，它所占用的内存空间也不会被释放

#变量的声明和定义

(所有的定义都是声明，但并非所有的声明都是定义)
声明：不分配内存
定义：分配内存
同一个编译单元里，同名的结构不能被重复声明

#程序运行步骤#

- 1.预处理：预处理器根据#开头的命令修改源程序，得到hello.i (处理宏)
- 2.编译阶段：将hello.i翻译成C编程语言程序hello.s
- 3.汇编阶段：将hello.s翻译为机器语言指令，并打包为一种可重定位目标程序的格式，结果保存在hello.o文件
- 4.链接阶段：将编译后形成的一组目标模块，以及所需库函数链接在一起，形成一个完整的装入模块hello，它是一个可执行目标文件
#链接有3种方式：
(1) 静态链接:在程序运行之前，先将各目标模块及它们所需的库函数连接成一个完整的可执行文件(装入模块)之后不再拆开，要运行时可直接将它装入内存。
(2) 装入时动态链接:将各目标模块装入内存时,边装入边链接的链接方式。
(3) 运行时动态链接:在程序执行中需要该目标模块时,才对它进行链接。其优点是便于修改和更新,便于实现对目标模块的共享。

注：编译阶段会针对静态变量、全局变量等做好内存分配的前期规划，链接阶段会对这些规划做进一步整合完善以便后续正确分配内存

#Makefile**

```
Makefile
1 CC = gcc
2 SRC = src
3 CFLAG = -I $(SRC)
4 OBJ5 = main.o $(SRC)/linkedlist.o
5
6 test: $(OBJ5)
7 $(CC) $(CFLAG) $(OBJ5) -o $@
8
9 .c.o:
10 $(CC) $(CFLAG) -c $< -o $@
11
12 clean:
13 rm -f $(OBJ5) test
14
```

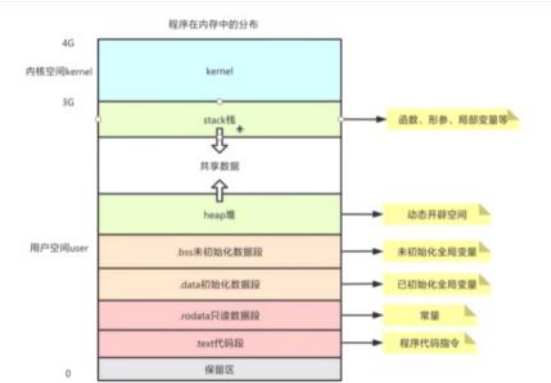
- 1.Makefile变量：
\$^ 表示上面的所有被依赖文件
\$@ 表示上边被依赖文件的目标
2.@ echo \$ ^ 表示输出上边所有被依赖文件
(@后边的echo不会被打出来，但是如果没有@就会被打出来)
3.代码 .o.c：等价于 %.o：%.c

##

预定义的宏：
__LINE__：表示所处的行号
__FILE__：文件名
__DATE__：编译时的日期
__TIME__：编译时的时间
__STD__：如果编译器遵循ANSI C，则其值为1，否则未定义

(汇总，不用记)
其余gcc命令行指令
gcc file.c：将 C 源文件编译为可执行文件。如果源文件中包含多个函数，则会自动链接必要的库文件
gcc -o output file.c：指定编译输出文件的名称
gcc -c file.c：将 C 源文件编译为目标文件，而不链接任何库文件
gcc -E file.c：预处理 C 源文件，并将结果输出到标准输出
gcc -Wall file.c：开启所有警告信息
gcc -g file.c：生成可以调试的可执行文件
gcc -O level file.c：开启优化选项，其中 level 的值可以是 0、1、2、3 或 s
gcc -I path file.c：添加一个包含文件搜索路径
gcc -L path file.c -l library：添加一个库文件搜索路径
gcc -D macro=value file.c：在编译时定义一个宏
注：可以通过输入 man gcc 来查看完整的帮助文档，或者通过输入 gcc --help 来查看常用选项的简介

#程序内存#



1.栈 (Stack) :

用于存储**局部变量**和函数调用的上下文信息, 如 **参数和返回地址**;

栈内存的申请和释放是由**编译器自动管理**的, 通常在函数调用时分配, 在函数返回时释放;

栈的大小通常有限, **过深的递归或大型局部变量可能导致栈溢出**。

2.堆 (Heap) :

堆是用于**动态内存分配**的内存区域, 可以通过malloc、free等函数显式地分配和释放内存;

堆内存的管理更加灵活, 但也更容易出错, 如 **内存泄漏和野指针问题**;

堆的大小通常比栈大得多, 但也需要程序员合理管理, 以避免内存碎片等问题。

3.全局/静态存储区:

用于存放全局变量和静态变量, 它们在程序的整个运行期间都存在;

全局变量和静态变量在编译时分配内存, 通常位于**数据段** (已初始化) 或**BSS段** (未初始化) 。

4. 常量存储区: 用于存放程序中的常量数据, 这些数据在程序运行期间 **不可修改**。

5. 代码段: 包含程序的执行代码, 这部分内存是只读的, 用于存储指令。

W15 数据类型

2024年12月29日 21:23

ASCII码表

ASCII值	控制字符	ASCII值	控制字符	ASCII值	控制字符	ASCII值	控制字符
0	NUT	32	(space)	64	@	96	`
1	SOH	33	!	65	A	97	a
2	STX	34	"	66	B	98	b
3	ETX	35	#	67	C	99	c
4	EOT	36	\$	68	D	100	d
5	ENQ	37	%	69	E	101	e
6	ACK	38	&	70	F	102	f
7	BEL	39	,	71	G	103	g
8	BS	40	(	72	H	104	h
9	HT	41	)	73	I	105	i
10	LF	42	*	74	J	106	j
11	VT	43	+	75	K	107	k
12	FF	44	,	76	L	108	l
13	CR	45	-	77	M	109	m
14	SO	46	.	78	N	110	n
15	SI	47	/	79	O	111	o
16	DLE	48	0	80	P	112	p
17	DC1	49	1	81	Q	113	q
18	DC2	50	2	82	R	114	r
19	DC3	51	3	83	S	115	s
20	DC4	52	4	84	T	116	t
21	NAK	53	5	85	U	117	u
22	SYN	54	6	86	V	118	v
23	TB	55	7	87	W	119	w
24	CAN	56	8	88	X	120	x
25	EM	57	9	89	Y	121	y
26	SUB	58	:	90	Z	122	z
27	ESC	59	;	91	[	123	{
28	FS	60	<	92	\	124	
29	GS	61	=	93	]	125	}
30	RS	62	>	94	^	126	~
31	US	63	?	95	_	127	DEL

二进制：
对于一个字节：
0000 0000——>0
1111 1111~1000 0000——>1~128
0000 0001~0111 1111——>1~127

```
#include <stdio.h>
int main() {
    int x;
    scanf("%x", &x);
    printf("%x, %d", x, x);
}
Input:18
Output:18,24
```

浮点数的运算结果不能直接比较是否相等
正确：

```
#include <math.h>
...
double a,b;
if(fabs(a-b)<1e-3)
...
```


大小写字母的转换
字母在 ASCII 表中按顺序排列
大写字母和小写字母是分开的
‘a’ - ‘A’ 得出大小写字母之间的间距，所以
a + ‘a’ - ‘A’ 将 a 从大写转换为小写
a + ‘A’ - ‘a’ 将 a 从小写转换为大写

W16 文件底层操作

2024年12月25日 15:32

#格式化输入输出

printf

返回值：输出的字符数

%[flags][width][.prec][hl]type

Flag	含义
-	左对齐
+	在前面放 + 或 -
(space)	正数留空 （前面加空格）
0	0填充
width 或 prec	含义
number	最小字符数 （包括小数点所有）
*	下一个参数是字符数
.number	小数点后的位数
.*	下一个参数是小数点后的位数

e.g.

printf (" %d " , a , b) ;

类型修饰	含义
hh	单个字节
h	short
l	long
ll	long long
L	long double

type	用于	type	用于
i 或 d	int	g	float
u	unsigned int	G	float
o	八进制	a 或 A	十六进制浮点
x	十六进制	c	char
X	字母大写的十六进制	s	字符串
f 或 F	float	p	指针
e 或 E	指数	n	读入/写出的个数

scanf

返回值：读入的项目数

%[flag]type

flag	含义	flag	含义
*	跳过	l	long , double
数字	最大字符数	ll	long long
hh	char	L	long double
h	Short		

type	用于	type	用于
d	int	s	字符串(单词)
i	整数，可能为十六进制或八进制	[...]	所允许的字符
u	unsigned int	p	指针
-	无修饰	-	无修饰

##与文件有关的函数

打开文件的标准代码

```
FILE* fp = fopen("file", "r");
if ( fp ) {
    fscanf(fp,...);
    fclose(fp);
} else {
    ...
}
```

##文件打开函数

fopen ()

mode	功能
r	打开只读
r+	打开读写，从文件头开始
w	打开只写。如果不存在则新建，如果存在则清空
w+	打开读写。如果不存在则新建，如果存在则清空
a	打开追加。如果不存在则新建，如果存在则从文件尾开始
a+	打开追加并读写
x	只新建，如果文件已存在则不能打开

*后边加上b为对二进制文件的操作

功能：

用于打开一个文件，并返回一个指向该文件的文件指针（后续进行文件操作的关键）

e.g.

FILE *fp;

fp = fopen("test.txt", "r");

// 以只读模式打开名为test.txt的文件；如果文件不存在，对于只读模式会返回NULL

##文件关闭函数

fclose ()

功能：关闭一个已经打开的文件

//这是一个很重要的操作，因为不关闭文件可能会导致数据丢失、文件损坏或者占用过多的系统资源。

e.g.

fclose(fp);

// 关闭之前打开的文件指针fp所指向的文件

##文件定位函数

fseek()、ftell()和rewind()

功能：

fseek()：用于设置文件位置指针的位置。可以将指针向前或向后移动指定的字节数，从而实现对文件的随机访问

ftell()：返回文件位置指针的当前位置，通常返回相对于文件开头的字节数

rewind()：将文件位置指针重新定位到文件的开头

e.g.

fseek(fp, 10L, SEEK_SET);

// 将文件fp的位置指针从文件开头（SEEK_SET）向后移动10个字节

SEEK_SET :从头开始

SEEK_CUR :从当前位置开始

SEEK_END :从尾开始(倒过来)

long pos = ftell(fp);

// 获取文件fp的位置指针当前位置

rewind(fp);

// 将文件fp的位置指针重新定位到文件开头

##文件结束判断函数

feof()

功能：

i	整数，可能为十六进制或八进制	[...]	所允许的字符
u	unsigned int	p	指针
o	八进制	x	十六进制
c	Char		

##文件结束判断函数

feof()

功能:

用于判断文件是否读取到了末尾

当文件位置指针到达文件末尾时，feof()函数返回真（1），否则返回假（0）

e.g.

```
while (!feof(fp)){
// 读取文件内容
}
```

##字符读取函数

fgetc()和getc()

功能:

从指定的文件中读取一个字符，使文件位置指针向后移动一个字符的位置

如果读取到文件末尾，会返回EOF

e.g.

int ch;

ch = fgetc(fp);

// 从文件fp中读取一个字符并存储到ch中。

##字符串读取函数

fgets()

功能:

从指定文件中读取一行字符串，直到遇到换行符'\n'或者读取了指定长度 - 1 个字符（最后一个字符要用来存储'\0'）//比较安全，可以有效防止缓冲区溢出。

e.g.

char str[100];

fgets(str, sizeof(str), fp);

// 从文件fp中读取一行字符串存放到str数组中

##格式化读取函数

fscanf()

功能:

按照指定的格式从文件中读取数据

e.g.

int num;

fscanf(fp, "%d", &num);

// 从文件fp中按照整数格式读取一个数据存储到num变量中

##字符写入函数

fputc()和putc()

功能:

将一个字符写入指定的文件中，同时文件位置指针会向后移动一个字符的位置

当写入操作成功时，它会返回写入的字符；如果写文件失败，fputc函数返回EOF（-1）

e.g.

char ch = 'A';

fputc(ch, fp);

// 将字符'A'写入文件fp中

##字符串写入函数

fputs()

功能:

将一个字符串写入指定的文件中，不包括字符串结尾的'\0'

e.g.

char str[] = "Hello, World!";

fputs(str, fp);

// 将字符串"Hello, World!"写入文件fp中

##格式化写入函数

fprintf()

功能:

按照指定的格式将数据写入文件

e.g.

```
int num = 10;  
fprintf(fp, "%d", num);  
// 将整数10按照整数格式写入文件fp中
```

常用函数

2024年12月27日 10:12

##绝对值函数:

```
<stdlib.h>
abs() : int
labs() : long int
llabs() : long long
int
```

```
<math.h>
fabs() : double
```

```
<math.h>
```

```
#pow
double pow(double base,
double exponent)
返回base的exponent次幂的值
```

```
#exp
double exp(double x)
返回e的x次幂的值
```

```
#sqrt
double sqrt(double b=num)
开方
```

##字符串和整数之间的转换

```
strtol(str, &a, 10);
(第三个参数表示十进制)
sprintf(str, "%d", num);
e.g.
```

```
char str2[] = "0x12";
long int num2 = strtol(str2, NULL, 0);
//base=0代表自动判断进制
printf("转换后的值: %ld\n", num2);
sprintf(str2, "%1o", num2);
printf("转换后的字符串: %s\n", str2);
```

##动态分配内存相关<stdlib.h>

1.malloc

```
void* malloc(size_t size);
```

用于在内存的堆区分配指定字节大小的连续内存空间

参数 size表示要分配的字节数

返回值是一个指向所分配内存空间起始地址的指针；如果分配失败（例如系统内存不足等原因），则返回 NULL

e.g.

```
int *ptr; ptr = (int *)malloc(5 * sizeof(int));
```

2.realloc

```
void* realloc(void* ptr, size_t size);
```

用于对已经分配的内存空间调整大小

参数ptr是之前分配内存的起始指针；参数size是调整后想要的内存大小（字节数）

如果调整成功，会返回指向新的（可能已经移动位置的）内存空间起始地址的指针；如果无法按照要求调整大小，则返回 NULL，但原内存空间不会被修改

e.g.

```
int *ptr = (int *)malloc(3 * sizeof(int));
```

3.calloc

```
void* calloc(size_t num, size_t size);
```

用于在堆区分配指定数量（num个）且指定大小（size个字节）的连续内存空间，并将分配的内存空间初始化为全 0

返回指向所分配内存起始地址的指针；如果分配失败，则返回 NULL

e.g.

```
int *arr; arr = (int *)calloc(3, sizeof(int));
```

4.free

```
void free(void* ptr);
```

用于释放之前通过 `malloc`、`calloc` 或 `realloc` 等函数分配的内存空间

运算符

2024年12月30日 20:21

优先级	运算符及其运算符	结合规律
1	[] () (结构体访问成员点方式) ->(结构体访问成员箭头方式)	从左向右
2	++ -- sizeof *(指针解引用) &(取地址) +/- (正负号) ~(按位取反) ！	从右向左
3	强制类型转换	从右向左
4	*(乘) /(除) %(取余)	从左向右
5	+ - (算术加减)	从左向右
6	<< >> (位移位)	从左向右
7	< <= >= > (小于 小于等于 大于等于 大于)	从左向右
8	== !=	从左向右
9	& (按位与)	从左向右
10	^ (按位异或)	从左向右
11	(按位或)	从左向右
12	&& (逻辑与)	从左向右
13	(逻辑或)	从左向右
14	? : (三目运算 如: a?1:0)	从左向右
15	= *= /= %= += -= <<= >>= &= ^= = (与赋值号相关的)	从右向左
16	, (逗号运算符)	从右向左

@陌生运算符解释

'!' 运算符#

在 C 语言中，0 代表假（false），非 0 值代表真（true）

当使用！运算符时，如果操作数为真，！运算后结果为假；如果操作数为假，！运算后结果为真

',' 运算符#

将多个表达式连接在一起，从左到右依次计算这些表达式的值

整个逗号表达式的值是最后一个表达式的值

#三目运算符（唯一）#

条件运算符？：

表达式1？表达式2：表达式3

先执行表达式1，如果表达式1为真，则执行表达式2，否则执行表达式3

#宏中使用的运算符

1.'#'运算符用于在宏定义中把宏参数转换为字符串常量

2.'##'运算符在宏定义中用于连接两个标识符（变量名、函数名、类型名等），形成一个新的标识符。

在创建具有不同命名的相关变量、函数等方面非常有用

##按位运算运算符

& 按位与

1&1——>1

其余——>0

应用：

让某些位为0：e.g.1001可让第2、3位为0

求余：e.g. x%2^n==x&(2n-1)

| 按位或

只要有一个1，则为1

应用：

让某些位为1

~ 按位取反

~1001=0110

^ 按位异或

相等——>1

不相等——>0

x^y=0——>x==y

x^y^y==x

<< 左移

i<<j：i中所有位向左移动j个位置，右边补0

x<<=n <=> x*=2^n

应用：

输出一个数的二进制

```
int number;scanf("%d", &number);for ( unsigned mask = 1u<<31; mask ; mask >= 1 ) {    printf("%d", number & mask?1:0);}
```

printf("\n");

W2 循环计算

2024年12月23日 20:48

'!' 运算符#

在 C 语言中，0 代表假（false），非 0 值代表真（true）
当使用！运算符时，如果操作数为真，！运算后结果为假；
如果操作数为假，！运算后结果为真

';' 运算符#

将多个表达式连接在一起，从左到右依次计算这些表达式的值
整个逗号表达式的值是最后一个表达式的值

##

else与最近的if并列

6-1 是不是倍数

实现一个函数，判断一个整数a是否是另一个整数b的倍数（指存在整数c，使得a==b*c）。

函数接口定义：

```
int isMultipleOf ( int a, int b );
```

其中a、b均不超过int范围。如果a是b的倍数，返回1，否则返回0。

裁判测试程序样例：

```
#include <stdio.h>
int isMultipleOf ( int a, int b );
int main() {
    int a, b;
    scanf("%d%d", &a, &b);
    printf("%d\n", isMultipleOf(a, b));
    return 0;
}
/* 请在这里填写答案 */
```

输入样例：

21 2

输出样例：

0

常见错误：相除后，int越界

答案：

```
int isMultipleOf ( int a, int b )
{
    long long int a1;
    a1=a;
    if (b==0&&a1==0){
        return 1;
    }
    else if (b==0&&a1!=0){return 0;}
    else{
        if (a1%b==0){return 1;}
        else{return 0;}
    }
}
```

7-3 整除光棍***

这里所谓的“光棍”，并不是指单身汪啦~ 说的是全部由1组成的数字，比如1、11、111、1111等。传说任何一个光棍都能被一个不以5结尾的奇数整除。比如，111111就可以被13整除。现在，你的程序要读入一个整数x，这个整数一定是奇数并且不以5结尾。然后，经过计算，输出两个数字：第一个数字s，表示x乘以s是一个光棍，第二个数字n是这个光棍的位数。这样的解当然不是唯一的，题目要求你输出最小的解。

提示：一个显然的办法是逐渐增加光棍的位数，直到可以整除x为止。但难点在于，s可能是个非常大的数——比如，程序输入31，那么就输出3584229390681和15，因为31乘以3584229390681的结果是1111111111111111，一共15个1。

输入格式：

输入在一行中给出一个不以5结尾的正奇数x（<1000）。

输出格式：

在一行中输出相应的最小的s和n，其间以1个空格分隔。

输入样例：

31

输出样例：

3584229390681 15

答案：

```
#include <stdio.h>
int main() {
    int x,n=1;
    int y,s;
    y=1;
    scanf("%d",&x);
    while(x>y){
        y=y*10+1;
        n+=1;
    } /**
    while (1){
        s=y/x;
        printf("%d",s);
        if(y%x==0){
            break;}
        n+=1;
        y=y*x*10+1;
    }
    printf(" %d",n);
}
```

W3 循环控制

2024年12月30日 9:56

##

Segment1:

```
if(x>=0)
if(x>0) y=1;
else y=0;
else y=-1;
```

Segment2:

```
if(x>0) y=1;
else if(x<0) y=-1;
else if(x=0) y=0;
```

以上两个片段得到的y的值相等

答案: F

解析: x=0时不相等, 第二个片段是x=0不是x==0

##

- ★ The following text is part of error information threw by the compiler when compiling some buggy C program.

ld: symbol(s) not found for architecture x86_64

clang: error: linker command failed with exit code 1

Which of the following problems exist in the C program is related to the information?

- A. Some variable is used without definition.
- B. A necessary semicolon ';' is missing.
- C. The program does not include the header file stdio.h but uses function printf.
- D. The only function in the program is named mian.

答案: D

W4 数组

2024年12月23日 22:43

##

sizeof()是运算符，不是函数

##

★ C语言中,通过函数调用只能获得一个返回值。

答案：F

解析：结构体，指针？

##

★ 在C语言程序中，若对函数类型未加显式说明，则函数的隐含类型为：

答案：int

##

若有以下调用语句，则不正确的 fun()函数的首部是（ ）。

main()

{ ...

int a[50], n;

...

fun(n, &a[9]);

...

}

A.void fun(int m, int x[])

B.void fun(int s, int h[41])

C.void fun(int p, int *s)

D.void fun(int n, int a)

答案：D

W5/W6 数组应用***

2024年12月23日 22:56

★ What will happen when you attempt to compile and run this C code?

```
#include <stdio.h>
(int, int) swap(int a, int b) {
    return (b, a);
}
int main()
{
    int a = 2, b = 6;
    printf("%d#%d", swap(a, b));
}
```

- A.It fails to compile
- B.It compiles but some runtime error occurs
- C.It compiles and prints out 2#6
- D.It compiles and prints out 6#2

答案: A

Among these definitions of 2-d arrays, which is/are correct?

- A.int a[1][2];
- B.int a[][2];
- C.int a[][2] = {0};
- D.int a[][2] = {0, 1, 2, 3, 4};
- E.int a[][2] = {{0, 1}, {2, 3},};
- F.int a[2][] = {{0, 1}, {2, 3}};

答案: ACDE

解析: 定义二维数组时, 不能省略列数, 不过在初始化时, 如果提供了足够的初始值, 可以省略行数, 编译器会根据初始值的个数来自动确定行数

7-7 出栈序列的合法性*****

给定一个最大容量为 m 的堆栈, 将 n 个数字按 1, 2, 3, ..., n 的顺序入栈, 允许按任何顺序出栈, 则哪些数字序列是不可能得到的? 例如给定 m=5、n=7, 则我们有可能得到{ 1, 2, 3, 4, 5, 6, 7 }, 但不可能得到{ 3, 2, 1, 7, 5, 6, 4 }。

输入格式:

输入第一行给出 3 个不超过 1000 的正整数: m (堆栈最大容量)、n (入栈元素个数)、k (待检查的出栈序列个数)。最后 k 行, 每行给出 n 个数字的出栈序列。所有同行数字以空格间隔。

输出格式:

对每一行出栈序列, 如果其的确是有可能得到的合法序列, 就在一行中输出 YES, 否则输出NO。

输入样例:

```
5 7 5
1 2 3 4 5 6 7
3 2 1 7 5 6 4
7 6 5 4 3 2 1
5 6 4 3 7 2 1
1 7 6 5 4 3 2
```

输出样例:

7-8 Gram-Schmidt Algorithm #####

The *Gram-Schmidt* algorithm is used to determine whether a set of n vectors $a_1, \dots, a_k$ are linearly independent.

Given n vectors $a_1, \dots, a_k$, let q_0 be the n -dimensional zero vector. For $i = 1, \dots, k$:

- Orthogonalization: $\tilde{q}_i = a_i - (q_1^T a_i)q_1 - \dots - (q_{i-1}^T a_i)q_{i-1}$
- Check for linear dependence: If $\tilde{q}_i = 0$, the algorithm terminates, indicating that the current vector is linearly dependent on the previous vectors.
- Normalization: $q_i = \frac{\tilde{q}_i}{\|\tilde{q}_i\|}$

For $i = 1$, since q_0 is zero, step 1 simplifies to $\tilde{q}_1 = a_1$. If the algorithm does not terminate early, then the set of vectors $a_1, \dots, a_k$ is linearly independent. Otherwise, if the algorithm terminates at $i = j$, i.e., $\tilde{q}_j = 0$, this means that a_j is a linear combination of $a_1, \dots, a_{j-1}$, and the set of vectors is lin-

early dependent.

Note that, due to floating-point computation errors, we consider $\epsilon = 10^{-6}$, which means two numbers are considered equal if their difference is less than $1e-6$.

Additionally:

Euclidean Norm

The Euclidean norm of an n -dimensional vector x , denoted $\|x\|$, is:

$\|x\| = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}$

It can also be expressed as the square root of the inner product of the vector with itself, $\|x\| = \sqrt{x^T x}$.

Inner Product (Dot Product)

The inner product (also known as dot product) of two n -dimensional vectors is defined as the following scalar:

$a^T b = a_1 b_1 + a_2 b_2 + \dots + a_n b_n$

This is the sum of the products of the corresponding elements. In some books, this is denoted as $a \cdot b$.

This problem's data and standard solution were provided by 张庆春.

Input Format:

The first line contains two integers N and M. N represents the dimension of the vectors, and M represents the number of vectors.

The next M lines provide the M vectors, represented by floating-point numbers, with each number separated by a space.

(For languages that distinguish between float and double, it is recommended to use double)

Output Format:

If the set of input vectors is found to be linearly independent, output NO, followed by the final q vector in the second line. Each value should be kept to two decimal places, with a space separating each number.

If the set of input vectors is found to be linearly dependent, output YES, followed by the index of the vector (starting from 1) at which the algorithm terminates.

Sample Input 1:

```
5 4
1 2 -1 -2 -3
```

5 6 4 3 7 2 1

1 7 6 5 4 3 2

输出样例:

YES

NO

NO

YES

NO

答案:

```
#include <stdio.h>
int main() {
    int m, n, k;
    scanf("%d %d %d\n", &m, &n, &k);
    int a[n];
    for (int i=0; i<k; i++) {
        for (int j=0; j<n; j++) {
            scanf("%d", &a[j]);
        }
        int max=0;
        int b[1000];
        int top=0;
        for (int x=0; x<n; x++) {
            for (int y=max; y<a[x]; y++) {
                max++; //max是栈内存储过的最大数
                b[top]=y+1; //b[]储存的是当前栈内的数据
                top++; //top是当前栈内的数据数量
            }
            if (top>m || b[top-1]>a[x]) {
                /*判断数据数量是否大于栈的容量;
                判断下一个出栈的元素是否大于当前元素 (当前元素能否出栈)
                e.g. 3 2 1 7 5 6 4*/
                printf("NO\n");
                goto Next;
            }
            else if (b[top-1]==a[x]) {top--;}
            //相当于a[x]出栈
        }
        printf("YES\n");
        Next;
    }
}
```

Sample Input 1:

5 4

1 2 -1 -2 -3

2 -1 3 -2 7

-1 3 2 4 28

3 10 -2 1 1

Sample Output 1:

NO

0.52 0.36 0.49 0.59 -0.14

Sample Input 2:

3 4

0.1 1 2

-0.3 0 -5

3 0 -0.3

0.2 -1 3

Sample Output 2:

YES 4

答案:

```
#include <stdio.h>
#include <math.h>
double nj(double a[], double b[], int n);
double euc(double a[], int n);
int main() {
    int n, m;
    scanf("%d %d\n", &n, &m);
    double a[m][n];
    for (int i=0; i<m; i++) { //输入
        for (int j=0; j<n; j++) {
            scanf("%lf", &a[i][j]);
        }
    }
    int ct=0, sum=0;
    double q[n], b[m][n];
    for (int i=0; i<n; i++) {
        q[i]=a[0][i];
        b[0][i]=q[i]/euc(a[0], n);
        if (q[i]>=1e-6 || q[i]<=-1e-6) {ct=1;}
    }
    for (int j=1; j<m; j++) {
        if (ct==0) {printf("YES 1"); sum++; break;}
        ct=0;
        for (int i=0; i<n; i++) {
            q[i]=a[j][i];
            for (int x=0; x<j; x++) {
                q[i]-=nj(b[x], a[j], n)*b[x][i];
            }
            if (q[i]>=1e-6 || q[i]<=-1e-6) {ct=1;}
        }
        if (ct==0) {
            printf("YES %d", j+1);
            break;
        }
        sum++;
        for (int y=0; y<n; y++) {
            b[j][y]=q[y]/euc(q, n);
        }
    }
    if (sum==m-1 && sum!=0) {
        printf("NO\n%.2lf", b[m-1][0]);
        for (int i=1; i<n; i++) {
            printf(" %.2lf", b[m-1][i]);
        }
    }
}
```

```
        else if(sum==0){printf("YES 1");}
    }
    double nj(double a[],double b[],int n){
        double nj=0;
        for(int i=0;i<n;i++){
            nj+=a[i]*b[i];
        }
        return nj;
    }
    double euc(double a[],int n){
        double euc=0;
        for(int i=0;i<n;i++){
            euc+=a[i]*a[i];
        }
        euc=sqrt(euc);
        return euc;
    }
}
```

W7 递归

2024年12月30日 10:10

##

函数内如果只有一条语句，可以没有大括号

答案：F

W8 搜索排序

2024年12月25日 11:57

##

What is the basic characteristic of quadratic complexity as the size of the problem doubles?

- A.The running time doubles
- B.The running time increases by a factor of four
- C.The running time is halved
- D.The running time remains the same

答案： B

解析：二次复杂度指的是 $O(n^2)$ ，当问题的复杂度翻倍时，变为 $O((2n)^2)=O(4n^2)$

##

Which partition scheme is more efficient because it does three times fewer swaps on average and creates efficient partitions even when all values are equal?

- A.Lomuto partition scheme
- B.Hoare partition scheme
- C.Quick sort partition scheme
- D.Merge sort partition scheme

答案： B

解析：平均而言，霍尔划分的交换次数要比洛穆托划分少大约三倍

##

What is the base case of the quicksort recursion?

- A.Arrays of size zero or one
- B.Arrays of size two
- C.Arrays of size three
- D.Arrays of any size

答案： A

解析：快排的基础情况是零个或一个元素的数组

##

When does the Lomuto partition scheme degrade to $O(n^2)$?

- A.When the array is already in order
- B.When the array is randomly shuffled
- C.When the array has only one element
- D.When the array has negative numbers

答案： A

解析：当元素已经逆序或正序的时候，快排最慢

6-1 九连环数列(递归版)

在国外的一本数学书中收录了各种各样的数列，其中就有这样一个数列：

1, 2, 5, 10, 21, 42, 85, 170, 341, ...

起先大家都感到莫名其妙，不知道它有什么用。它既非等差数列，又非等比数列，也不是一些非常有名的数列。后经人指点，大家才恍然大悟，原来它是九连环数列。

第 1 项为 1，表明解开 1 个环需 1 步；第 2 项为 2，表明解开 2 个环需要 2 步；.....，第 9 项为 341，表明解开 9 个环需要 341 步，.....，以此类推。请编写递归函数，求九连环数列中任一项的值。

函数原型

double NumRing(int index);

说明：参数 index 为索引号(正整数)，函数值为汉诺塔数列第 index 项的值。

要求：不要使用循环语句。不要调用 pow、exp 等函数。可调用前面作业中的 IsOdd 或 IsEven 函数判断奇数或偶数。

裁判程序

```
#include <stdio.h>
int IsOdd(int x);
int IsEven(int x);
double NumRing(int index);
int main()
{
    int n;
    scanf("%d", &n);
    printf("%.15g\n", NumRing(n));
    return 0;
}
/* 你提交的代码将被嵌在这里 */
```

输入样例1

5

输出样例1

21

输入样例2

6

输出样例2

42

输入样例3

45

输出样例3

23456248059221

解析：卸下一个环或者上一个环时，前一个环必须在上边；

卸下n个环，需要先解掉前n-2个环（an-2），然后摘下第n个，再上前n-2个，最后解下前n-1个

答案：

```
double NumRing(int index){
    double num=1;
    if(index==1){
        num=1;
    }
    else{
        if(IsOdd(index)==0){
            num=2*NumRing(index-1);
        }else{
```

```
        num=2*NumRing(index-1)+1;
    }
}
return num;
}
```

W10 指针

2024年12月27日 16:15

##

关于C语言指针的运算：指针只有加减操作，没有乘除操作。指针可以加常数、减常数；相同类型的指针可以相加、相减。

答案：F

解析：相同类型的指针不可以相加

##

- ★ 假设有定义如下： `int array[10]`；则该语句定义了一个数组array。其中array的类型是整型指针（即： `int *`）。

答案：F

解析：数组名 `array` 被隐式转换为 `int *` 类型，即指向 `array[0]` 的指针，但这并不意味着 `array` 本身就是指针类型。`array` 是一个数组类型，更准确地说，它的类型是 `int [10]`，即包含 10 个 `int` 类型元素的数组

数组名不能++或--

##

数组的基地址是在内存中存储数组的起始位置，数组名本身就是一个地址即指针值。

答案：T

##

变量定义： `int *p, q`；中， `p`和`q`都是指针

答案：F

##

- ★ Among the following assignments or initializations, __ is wrong.
A. `char str[10]; str="string";`
B. `char str[]="string";`
C. `char *p="string";`
D. `char *p; p="string";`

答案：A

解析： `char str[10]="string";`正确

##

- ★ According to the declaration: `int (*p)[10]`;, `p` is a(n) __.
A. pointer
B. array
C. function
D. element of array

答案：A

解析：数组指针

#指针数组： `int* p[5]`; (等价于 `int *(p[5])`)

##**

- ★ ? For the function declaration `void f(char ** p)`, the definition __ of var makes the function call `f(var)` incorrect.
A. `char var[10][10];`
B. `char *var[10];`
C. `void *var = NULL;`
D. `char *v=NULL, **var=&v;`

答案：A (C?)

解析：A. `char (*)[10]`类型

##

What is the significance of the 0 address in relation to pointers?

A. It is a normal address that pointers can freely use.

B. It can be used to represent special things like an invalid returned pointer or an uninitialized pointer.

C. It is used for pointer arithmetic operations.

D. It is used to indicate the end of an array when using pointers

答案：B

解析：在使用指针遍历数组时，判断数组结束往往是通过与指向数组最后一个元素的下一个位置的指针（通常是通过起始指针加上数组长度来得到）进行比较等方式来确定的，而不是用地址 0 来指示数组的结束

##

★ 以下哪个定义中的p不是指针，请选择恰当的选项：

A.char **p;

B.char (*p)[10];

C.char *p[6];

D.给出的三项中，p都是指针

答案：C

解析：元素是指针

##

★ 若有定义char *str[]={ "Python" , "SQL" , "JAVA" , "PHP" , "C++" };
则表达式*str[1] > *str[3]比较的是：

A.字符P和字符J

B.字符串SQL和字符串PHP

C.字符串Python和字符串JAVA

D.字符S和字符P

答案：D

W12 结构

2024年12月30日 18:03

##

What is the internal type of an enum?

A.int

B.pointer

C.struct

D.can be defined as any type

答案：A

解析：枚举

W13 链表

2024年12月30日 18:34

##

When implementing a linked list to store a polynomial, why might a linked list be preferred over an array?

A.A linked list requires less memory than an array.

B.A linked list can dynamically grow and shrink, making it suitable for sparse polynomials.

C.A linked list provides faster access to elements compared to an array.

D.A linked list is easier to implement than an array.

答案: B

##

约瑟夫环问题: 使用循环链表跟踪剩余的人, 并在人员被淘汰时删除节点

秦九韶算法: 多项式的嵌套计算

W14 程序结构

2024年12月16日 17:29

@判断题

预处理命令的前面必须加一个“#”号。
答案: T

★ 若有宏定义: #define S(a,b) t=a;a=b;b=t 由于变量t没定义, 所以此宏定义是错误的。
答案: F
解析: 宏定义没有问题, 如果后续没有定义t, 则会出现编译错误

宏定义不存在类型问题, 宏名无类型, 它的参数也无类型。
答案: T
解析: 实现原理、作用域、可变性等方面均不同

全局变量不可以和函数内的局部变量同名。
答案: F
解析: 全局变量的作用域从它被定义的位置开始, 到文件末尾都是有效的 (可以跨多个函数被访问), 但如果在某个函数内部存在和它同名的局部变量, 那么在该局部变量的作用域内 (即该函数内部), 同名的全局变量会被 “暂时遮蔽”

@选择题

★ 以下是一个C语言程序的除标准库之外的全部源代码, 则说法正确的是

```
#include <stdio.h>
extern int k;
int main() {
    k = 2223;
    printf("%d\n", k);
    return 0;
}
```

A.这段程序编译错误
B.这段程序编译正确, 但是链接 (link) 错误
C.这段程序编译、链接 (link) 正确, 但是运行时错误
D.程序无错, 可正常运行
答案: B
解析: extern int k; 声明了变量 k 是在其他地方定义的全局变量; 在整个提供的代码中, 并没有找到变量 k 的实际定义
链接阶段: 链接器负责将各个目标文件和库文件链接成一个可执行文件。在链接时, 它会去寻找变量 k 的实际定义, 由于在整个代码中都没有 k 的定义, 链接器无法完成链接, 会报链接错误, 提示找不到 k 的定义

★ 有如下多文件组织:

```
header.h
#ifndef HEADER_H
#define HEADER_H
char school[] = "Sanben";
void fun(char* s);
#endif
File1.c
#include "header.h"
int main()
{
    fun(school);
    printf("%s",school);
}
File2.c
#include "header.h"
#include <string.h>
void fun(char* s)
{
    strcpy(s, "Yiben");
    return;
}
```

程序输出结果为:
A.Sanben
B.Yiben
C.编译错误
D.链接错误
答案: D
解析: 没有头文件<stdio.h>, 不能使用printf

★ C语言的全局变量的初始化是在以下哪个阶段完成的:

A.main()函数开始后
B.编译链接的时候
C.main()函数开始前
D.第一次用到的时候
答案: C
解析:
编译阶段: 编译器会处理全局变量的声明以及相关的语法检查等工作。对于初始化表达式, 编译器会 **计算并确定其初始值的具体内容**, 但此时并不会真正去执行初始化操作。
链接阶段: 链接器主要负责将各个目标文件以及所需的库文件组合到一起, 形成一个完整的可执行文件。不涉及对全局变量实际的初始化
程序启动阶段 (main 函数开始前): 程序开始运行时, 在进入 main 函数之前, 会有一个专门的初始化过程来 **对全局变量进行初始化**。操作系统加载可执行文件到内存后, 会先按照之前编译时确定的初始值对全局变量进行赋值, 完成初始化操作, 之后才会开始执行 main 函数

int x = 5, y = 7;
void swap ()
{
 int z;
 z = x;
 x = y;
 y = z;
}
int main(void)
{
 int x = 3, y = 8;
 swap ();
 printf ("%d,%d \n", x, y);
 return 0;
}
正确的输出为 (3,8)
解析: swap在使用时未传入参数, 使用的是全局变量

★ 凡是函数中未指定存储类别的局部变量, 其隐含的存储类型为()。

A.自动 (auto)
B.静态 (static)
C.外部 (extern)
D.寄存器 (register)
答案: A
解析: 自动变量的特点是在函数被调用时才会为其分配内存空间, 函数执行结束后, 其所占用的内存空间就会被自动释放

★ 在一个C源程序文件中, 若要定义一个只允许本源文件中所有函数使用的全局变量, 则该变量需要使用的存储类别是
答案: static

W15 数据类型

2024年12月29日 22:00

##

★ 文件中定义的全局变量的作用域:

- A.本程序的全部范围
- B.本文件的全部范围
- C.函数内的全部范围
- D.从定义该变量的位置开始到本文件结束

答案: D

##

下面说法中错误的是

- A.若全局变量仅在单个C文件中访问,则可以将这个变量修改为静态全局变量,以降低模块间的耦合度
- B.静态全局变量使用过多,可那会导致动态存储区(堆栈)溢出
- C.设计和使用访问动态全局变量、静态全局变量、静态局部变量的函数时,需要考虑变量生命周期问题
- D.若全局变量仅由单个函数访问,则可以将这个变量改为该函数的静态局部变量,以降低模块间的耦合度

答案: B

解析: 静态全局变量不存储在动态存储区内

##

★ 以下程序的输出的结果是:

```
#include<stdio.h>
int x=3;
void incre();
int main()
{
    int i;
    for (i=1;i<x;i++)  incre();
    return 0;
}
void incre()
{
    static int x=1;
    x*=x+1;
    printf("  %d",x);
}
```

- A.3 3
- B.2 6
- C.2 2
- D.2 5

答案: B

解析: 静态局部变量

7-5 研究: 双精度实数的机内码*

请编写程序,输入十进制双精度实数,输出其 64 位机内码。

要求: 以十六进制形式输出机内码。

输入格式

十进制实数

输出格式

机内码(十六进制)

要求: 十六进制机内码中的字母均为大写。

例如: 实数 3.6 转换成二进制是 11.1001100110011001..., 科学计数法记为: 1.11001100110011001...x

101, 因此 64 位的机内码为:

0100000000001100110011001100110011001100110011001100110011001101, 用十六进制书写则为:

400CCCCCCCCCCD

样例输入

3.6

样例输出

400CCCCCCCCCCD

答案:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
int main() {
    double n;
    scanf("%lf",&n);
    char *m;
    m=(char *)malloc(sizeof(char)*65);
    if(n>=0) {
        m[0]='0';
    }else{
        m[0]='1';
    }
    int n1=(int)n;
    double n2=n-n1;
    int cnt=0;
    while(n1!=0) {
        n1/=2;
        cnt++;
    }
    n1=(int)n;
    int num=cnt-1;
    cnt+=1022;
    for(int i=11;i>0;i--) {
        if(cnt%2==0) {
            m[i]='0';
        }else{
            m[i]='1';
        }
        cnt/=2;
    }
    for(int i=12+num-1;i>11;i--) {
        if(n1%2==0) {
            m[i]='0';
        }else{
            m[i]='1';
        }
        n1/=2;
    }
}
```

```
for(int i=12+num;i<64;i++){
    int x=(int) (n2*2);
    if(x==0){
        m[i]='0';
    }else{
        m[i]='1';
    }
    n2=n2*2-x;
}
long long int s=strtoll(m,&m+63,2);
printf("%11X\n",s);
}
```

W16 文件底层操作

2024年12月25日 16:01

★ @判断题

##

文件指针和位置指针都是随着文件的读写操作在不断改变

答案：F

解析：文件指针不改变，代表这个文件在内存中的访问入口

##

随机操作只适用于文本文件

答案：F

解析：

随机操作是指可以在文件的任意位置进行读写操作，而不是只能顺序地从文件开头依次进行读写。它主要依赖于文件内部的位置指针，通过改变位置指针的值，就可以跳到文件的指定位置进行读写。比如fseek的参数：

SEEK_SET :从头开始

SEEK_CUR :从当前位置开始

SEEK_END :从尾开始(倒过来)

适用于多种文件类型

##

文件的读函数是从输入文件中读取信息，并存放在内存中。

答案：T

@选择题

##

如果二进制文件a.dat已经存在，现在要求写入全新数据，应以____方式打开

A."w"	B."wb"	C."w+"	D."wb+"
-------	--------	--------	---------

答案：B

##

按数据的组织形式划分，文件可以分为：

A.记录文件和流式文件

B.普通文件和设备文件

C.文本文件和二进制文件

D.程序文件和数据文件

答案：C

##

以下可作为函数fopen中第一个参数的正确格式是（ ）

A.c:user\text.txt

B.c:\user\text.txt

C."c:\user\text.txt"

D."c:\\user\\text.txt"

答案：D

解析：fopen的第一个参数是字符串，需要用“ ”

##

函数fgetc的作用是从指定文件读入一个字符，该文件的打开方式可以是

A.只写

B.追加

C.读或读写

D.答案B和C都正确

答案：D

解析：

fopen中使用a模式打开的文件可以读，但需要先调整文件位置指针才能有效地读取文件内容
当文件以a模式打开时，文件位置指针会自动移动到文件的末尾

程序题常见错误

2024年12月23日 21:00

#浮点错误:

1.除数为0

##注:

1.定义变量的时候一定记得初始化

2.当输入的个数不定时,可以利用scanf的返回值-1来终止循环

注意点

2024年12月30日 20:43

```
#include <stdio.h>
int main() {
    int num = 5;
    printf("%d %d\n", num, ++num);
    return 0;
}
```

在某些编译器（比如常见的gcc编译器遵循从右到左的求值顺序）中，对于printf函数里的参数，会先计算最右边的++num。执行++num时，num的值先自增为6，然后再计算左边的num，此时num的值因为前面的自增操作已经变成了6，所以最终输出的结果是6 6。

然而，如果编译器按照从左到右的顺序求值参数，那么会先处理左边的num，此时它的值是5，接着执行++num，num自增为6，最终输出的结果可能就是5 6。

English

2024年12月28日 18:04

Exponent	(n.) 指数	subtraction	(n.) 减法
Coefficient	(n.) 系数	addition	(n.) 加法
adjacent	(adj.) 临近的	increment	(n.) 自增
retrieve	(v.) 获取, 检索	decrement	(n.) 自减
via	(prep.) 通过, 借助	multiplication	(n.) 乘法
access	(v.) 访问	division	(n.) 除法
prototype	(n.) 原型	shuffle	(v.) 打乱, 搅乱
parameter	(n.) 参数	concatenate	(v.) 连接
scope	(n.) 范围	implement	(v.) 实现
unary	(adj.) 一元的	polynomial	(n.) 多项式
Assignment operator	赋值运算符	dynamical	(adj.) 动态的
expression	表达式	nest	(v.) 嵌套
compiler	(n.) 编译程序	sequentially	(adv.) 依次地
status	(n.) 状态	macro	(n.) 宏
Segment	(n.) 片段	punch	(n.) #
Declaration	(n.) 声明	skeleton	(n.) 骨架, 框架
implicit	(adj.) 隐含的, 暗示的	Stack	(n.) 栈
invalid	(adj.) 无效的	heap	(n.) 堆
iterate	(v.) 重复, 迭代		
eliminate	(v.) 消除, 去除		
quadratic	(adj.) 二次的, 平方的		
computational	(adj.) 计算的		
arithmetic	(adj.) 算术的		